DEVOPS PROJECT PORTFOLIO - Andres Cepeda

INTRODUCTION

This document presents a complete DevOps hands-on project where a cloud-based web application was designed, deployed, secured, and troubleshooted using AWS and modern DevOps tools.

ARCHITECTURE DIAGRAM

Internet -> HTTPS (ngrok) -> Nginx (port 80) -> Gunicorn (127.0.0.1:5000) -> Flask Application

PROJECT STORY

The project started by provisioning an AWS EC2 Linux instance. After configuring SSH access, a web server (Nginx) was installed, followed by building a Flask application that serves a personal CV website. The application was upgraded to production grade using Gunicorn and exposed to the internet through a reverse proxy and HTTPS tunnel.

KEY COMPONENTS

AWS EC2: Compute instance running Linux SSH: Secure remote access WinSCP/SCP: File transfer Nginx: Reverse proxy and web server Flask: Web application framework Gunicorn: Production WSGI server systemd: Service management ngrok: Secure HTTPS exposure

DEPLOYMENT FLOW

1. Create EC2 instance 2. Connect via SSH 3. Install Nginx 4. Deploy Flask app 5. Run with Gunicorn 6. Configure reverse proxy 7. Enable systemd service 8. Expose via ngrok

TROUBLESHOOTING EXPERIENCE

Resolved real-world issues including: - Private vs public IP access - Port conflicts (5000 already in use) - Binding issues (localhost vs 0.0.0.0) - DNS resolution errors (ngrok) - systemd service configuration - SSH key and path issues

SECURITY PRACTICES

Only port 80 exposed publicly. Internal services are isolated. SSH secured with key authentication. Reverse proxy hides backend services.

GITHUB README STRUCTURE

This project includes a professional README with architecture, commands, troubleshooting, and DevOps practices.

INTERVIEW SUMMARY

Deployed a production-ready Flask application on AWS EC2 using Nginx as reverse proxy and Gunicorn as server, managed with systemd and exposed via HTTPS using ngrok.

FUTURE IMPROVEMENTS

Docker containerization CI/CD pipelines Custom domain with SSL Deployment automation